



COMILLAS
UNIVERSIDAD PONTIFICIA

ICAI

ICADE

CIHS

Sprint 6

Grupo:

Jorge Carnicero Príncipe,

Andrés Gil Vicente,

Jorge González Pérez

Tecnologías de Procesamiento Big Data

3º Grado en Ingeniería Matemática e Inteligencia Artificial

Índice

1. Introducción.....	3
1.1. Contexto.....	3
1.2. Objetivos del Sprint 6.....	3
1.3. Justificación.....	3
2. Metodología.....	5
2.1. Respuesta a HU-8: Persistencia y Visualización en Tiempo Real.....	5
2.2. Consumo de los Topics de Kafka.....	5
2.3. Escritura en Amazon Timestream.....	6
2.4. Visualización en Grafana.....	7
3. Resultados.....	8
3.1. Descripción de los Resultados Obtenidos.....	8
3.2. Validación de la Escritura en Timestream y del Dashboard de Grafana.....	8
4. Conclusión.....	10
4.1. Resumen del Proceso.....	10
4.2. Principales Logros.....	10

1. Introducción

1.1. Contexto

El Sprint 6 cierra el ciclo completo de la arquitectura de procesamiento streaming del proyecto TradeData, añadiendo las dos últimas etapas que faltaban: el almacenamiento persistente de los datos en tiempo real y su visualización en un dashboard interactivo. En los sprints anteriores hemos construido un pipeline operativo de extremo a extremo que realiza este proceso: en el Sprint 5.1 implementamos la adquisición de datos en tiempo real desde Binance para el par *SOLUSD* y su publicación en el topic de Kafka *imat3a_SOL_BigDaddyks*; en el Sprint 5.2 desarrollamos el job de AWS Glue, de tipo Spark Structured Streaming que consume ese topic, calcula el VWAP por ventanas de 5 minutos y publica los resultados en el topic *imat3a_SOL_BigDaddyks_VWAP*.

Con ambos topics produciendo datos en tiempo real, en el Sprint 6 respondemos a la pregunta que quedaba pendiente: ¿dónde se almacenan estos datos y cómo se visualizan?

La respuesta la proporciona la HU-8 a través de dos herramientas. Amazon Timestream actúa como base de datos orientada a series temporales administrada por AWS, recibiendo los registros de ambos topics y almacenando dichos registros de forma optimizada para consultas temporales. Por su parte, Grafana es una plataforma de visualización y análisis de datos open source, que consume esos datos directamente desde Timestream y los presenta en un dashboard interactivo que permite monitorizar en tiempo real las cotizaciones persistentes y el VWAP calculado sobre el flujo streaming.

Este sprint completa así la secuencia completa que las diapositivas de la asignatura definen para la arquitectura de procesamiento streaming:

Binance WebSocket → Python Producer → Kafka → Spark Streaming → Kafka → Timestream → Grafana.

1.2. Objetivos del Sprint 6

Los objetivos concretos del Sprint 6, tal y como se recogen en la HU-8, son los siguientes:

- Implementar un script en Python que consuma los mensajes de los dos topics de Kafka (*imat3a_SOL_BigDaddyks* e *imat3a_SOL_BigDaddyks_VWAP*) y almacene los datos en Amazon Timestream.
- Escribir los datos en dos tablas de Amazon Timestream, una para las cotizaciones en crudo (close por vela de 1 minuto) y otra para los resultados del VWAP (por ventanas de 5 minutos, cada 1 minuto).
- Conectar Grafana con Amazon Timestream y construir un dashboard que visualice en tiempo real los datos almacenados en ambas tablas.

1.3. Justificación

La necesidad de añadir una capa de persistencia responde a una limitación de Kafka como sistema de mensajería: aunque proporciona una capa de mensajería y almacenamiento intermedio adecuada para el flujo streaming; para la consulta temporal y la visualización mediante dashboards resulta conveniente incorporar una capa específica de almacenamiento y visualización.

Para poder visualizar la evolución temporal de los datos y construir dashboards interactivos es necesario volcarlos en un sistema de almacenamiento optimizado para ese fin.

Amazon Timestream es la elección natural dentro del ecosistema AWS para este caso de uso. Las diapositivas de la asignatura lo describen como un servicio de base de datos para series temporales completamente administrado, diseñado específicamente para recopilar, almacenar y analizar datos que varían con el tiempo, y destacan que puede ser más rápido y económico que las bases de datos relacionales tradicionales en este tipo de escenarios. Su compatibilidad nativa con Grafana elimina además la necesidad de capas intermedias de transformación para la visualización de los datos.

Grafana, por su parte, es la herramienta de visualización que las diapositivas presentan para el cierre de la arquitectura streaming. Su capacidad para conectarse con múltiples fuentes de datos, entre ellas Amazon Timestream, crear dashboards interactivos y generar alertas lo convierte en la mejor opción para monitorizar en tiempo real, que es justo lo que queremos.

2. Metodología

2.1. Respuesta a HU-8: Persistencia y Visualización en Tiempo Real

La HU-8 establece que debemos consumir los mensajes de los dos topics generados en los sprints anteriores, almacenarlos en Amazon Timestream y visualizarlos en Grafana.

Nosotros hemos optado por implementar un único script Python (*timestream.py*) que centraliza el consumo de ambos topics y gestiona la escritura en Timestream de forma diferenciada según el origen del mensaje, en lugar de crear dos scripts independientes. Esta decisión nos simplifica el despliegue del sistema y permite una lógica de consumo unificada.

Este script actúa como el último eslabón del pipeline: consume lo que el producer del Sprint 5.1 publica en el topic de velas y lo que el Glue job de Spark del Sprint 5.2 publica en el topic de VWAP, y escribe cada tipo de dato en su tabla correspondiente de Timestream. Grafana consulta directamente esas tablas para construir el dashboard de visualización en tiempo real.

2.2. Consumo de los Topics de Kafka

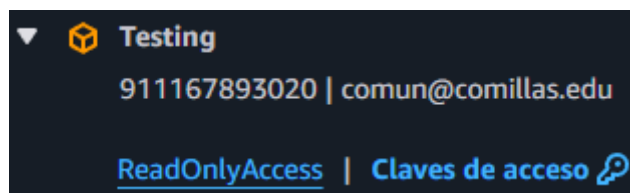
El script *timestream.py* instancia un único *KafkaConsumer*, en coherencia con la configuración utilizada en el pipeline de Kafka de los sprints anteriores, y se suscribe simultáneamente a ambos topics mediante el método *subscribe*, de forma análoga a como lo hace el consumer del Sprint 5.2. Los parámetros de conexión son los mismos que en sprints anteriores:

- Bootstrap server: *51.49.235.244:9092*
- Protocolo de seguridad: *SASL_PLAINTEXT* con mecanismo *PLAIN*
- Usuario: *kafka_client*
- Group ID: *imat3a_SOL_BigDaddyks*
- *auto_offset_reset*: *latest*

El bucle principal realiza polls con un timeout de 1 segundo, procesando cada lote de mensajes en función del topic de origen. Para los mensajes del topic *imat3a_SOL_BigDaddyks* se invoca la función *build_quote_record*, y para los del topic *imat3a_SOL_BigDaddyks_VWAP* se invoca *build_vwap_record*. Cualquier excepción en el procesamiento o escritura de un registro individual queda capturada y registrada en consola sin detener el bucle, lo que garantiza la continuidad del consumo ante errores puntuales.

2.3. Escritura en Amazon Timestream

Amazon Timestream se configura en la región eu-west-1, el cliente de Timestream se inicializa mediante boto3, que resuelve las credenciales de AWS de forma automática a partir de las variables de entorno que le introducimos por terminal, y que podemos conseguirlas en el apartado de cuentas de la interfaz de AWS, en la cuenta de Testing:



La base de datos utilizada es *imat3a_crypto_rt*, y dentro de ella se escriben dos tablas diferenciadas:

Databases (1) Info	
search table	
Name	Creation time (UTC)
imat3a_crypto_rt	3/24/2026, 9:59:34 AM

Tabla *sol_quotes_raw_bigdaddyks*: recibe los registros de cotización en crudo procedentes del topic *imat3a_SOL_BigDaddyks*. Cada registro se construye mediante la función *build_quote_record* con la siguiente estructura:

- Dimensiones: *symbol* (procedente del campo *symbol* del JSON de entrada) y *source_topic* (nombre del topic de origen).
- Medida: el valor de close, casteado a Double desde el valor string que publica el producer.
- Timestamp: se extrae del campo *@timestamp* del mensaje JSON, convirtiéndolo de formato *ISO 8601 UTC* a milisegundos epoch mediante la función *iso_to_epoch_ms*. Si el campo no está presente, se usa el timestamp del propio mensaje de Kafka como fallback.
- Versión: timestamp de escritura en milisegundos.

sol_quotes_raw_bigdaddyks	imat3a_crypto_rt
---	----------------------------------

Tabla *sol_vwap_5m_bigdaddyks*: recibe los registros de VWAP procedentes del topic *imat3a_SOL_BigDaddyks_VWAP*. Cada registro se construye mediante la función *build_vwap_record* con la siguiente estructura:

- Dimensiones: *symbol*, *window_start*, *window_end* y *source_topic*.
- Medida: *vwap*, casteado a Double.
- Timestamp: se extrae del campo *window_end* del mensaje. Si *window_end* no está disponible, se usa el timestamp de Kafka como fallback.
- Versión: *timestamp* de escritura en milisegundos.

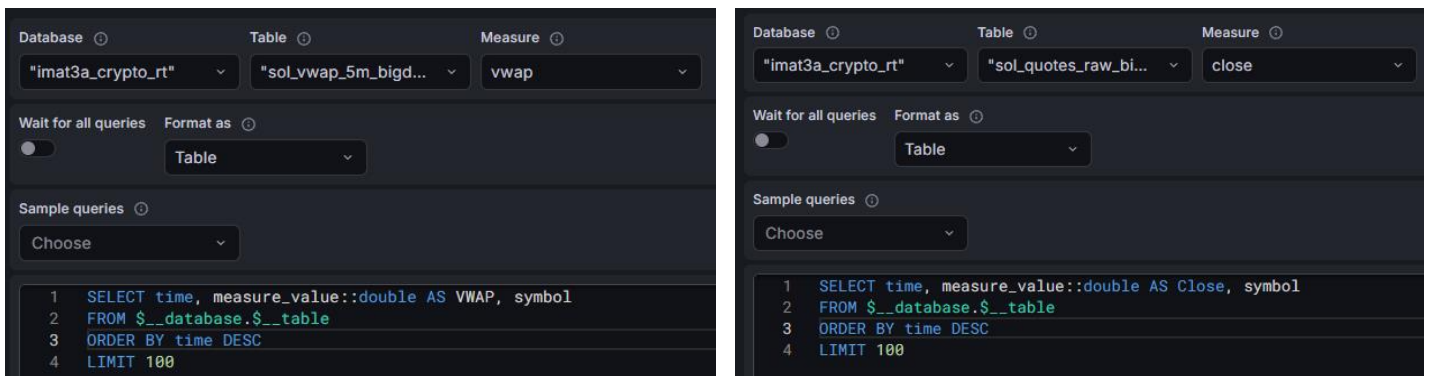
sol_vwap_5m_bigdaddyks	imat3a_crypto_rt
--	----------------------------------

La escritura en Timestream se realiza registro a registro mediante la llamada `ts.write_records`, que recibe el nombre de la base de datos, el nombre de la tabla y una lista con el registro a insertar. Cada vez que escribe se manda *"Write OK"* por terminal seguido del contenido, para monitorizar el correcto funcionamiento de todo el flujo.

2.4. Visualización en Grafana

Para la visualización de los resultados, hemos configurado Grafana accediendo a su web con las credenciales proporcionadas en el Excel de Moodle. Grafana se conecta a Amazon Timestream, apuntando a la base de datos *imat3a_crypto_rt* en la región eu-west-1.

El dashboard construido incluye los siguientes paneles:



3. Resultados

3.1. Descripción de los Resultados Obtenidos

Con el pipeline completo en ejecución (producir del Sprint 5.1 publicando cada minuto las velas cerradas en el topic `imat3a_SOL_BigDaddyks`, job de Spark del Sprint 5.2 calculando el VWAP con una ventana de 5 minutos, y script `timestream.py` consumiendo ambos topics) los datos fluyen a Amazon Timestream y quedan disponibles para consultarlos y visualizarlos en Grafana una vez se mandan a la base de datos.

La tabla `sol_quotes_raw_bigdaddyks` recibe aproximadamente un registro por minuto, correspondiente a cada vela cerrada del flujo de cotización en crudo consumido desde el topic de entrada, con el precio de cierre y el timestamp de evento extraído del campo `@timestamp` del mensaje. La tabla `sol_vwap_5m_bigdaddyks` recibe registros con una frecuencia que depende del trigger de Spark (actualmente la hemos puesto en 1 minuto) y del outputMode update.

El dashboard de Grafana que mostramos a continuación, refleja la evolución temporal de los valores del close de las cotizaciones almacenadas y del VWAP calculado sobre ventanas de 5 minutos, completando así el flujo de visualización que la arquitectura propuesta en las diapositivas de la asignatura define como objetivo final del proyecto streaming.

3.2. Validación de la Escritura en Timestream y del Dashboard de Grafana

La correcta escritura en Timestream se verifica mediante los mensajes `"Write OK"` que el script imprime en consola para cada registro insertado, mostrando el contenido completo del registro en formato JSON. Esto permite confirmar en tiempo real que los valores de close y vwap se están persistiendo correctamente con sus timestamps y dimensiones asociadas.

Además, veamos como se ven los resultados en el dashboard de Grafana:



— VWAP — CLOSE —

De este modo hemos verificado que:

- Los registros de la tabla `sol_quotes_raw_bigdaddyks` tienen el timestamp de evento correcto, derivado del campo `@timestamp` del mensaje de entrada consumido desde Kafka, que a su vez proviene del timestamp de cierre de la vela en Binance.
 - Los registros de la tabla `sol_vwap_5m_bigdaddyks` tienen el timestamp de evento asociado al `window_end` de cada ventana, lo que los sitúa correctamente en la serie temporal en el momento en que la ventana de 5 minutos concluye.
 - Grafana consulta correctamente ambas tablas de Timestream y representa los datos del pipeline en el dashboard configurado, mostrando todos los datos en conjunto correctamente.
-

4. Conclusión

4.1. Resumen del Proceso

En este sprint hemos implementado el script `timestream.py`, que consume de forma continua los mensajes de los topics `imat3a_SOL_BigDaddyks` e `imat3a_SOL_BigDaddyks_VWAP`, transforma cada mensaje en un registro de serie temporal y lo escribe en su tabla correspondiente de la base de datos `imat3a_crypto_rt` en Amazon Timestream (región eu-west-1). Las cotizaciones en crudo se almacenan en la tabla `sol_quotes_raw_bigdaddyks` con la medida `close`, y los resultados del VWAP de ventanas de 5 minutos se almacenan en la tabla `sol_vwap_5m_bigdaddyks` con la medida `vwap`. Sobre esos datos hemos construido un dashboard en Grafana que visualiza en tiempo real la evolución de ambos indicadores, cerrando el ciclo completo de la arquitectura streaming del proyecto.

4.2. Principales Logros

Los logros principales que hemos conseguido en el Sprint 6 incluyen:

1. Pipeline streaming completo end-to-end: El proyecto cuenta ahora con un flujo completamente operativo desde Binance hasta Grafana, pasando por Kafka, Spark Structured Streaming y Amazon Timestream, tal y como define la arquitectura propuesta en las diapositivas de la asignatura.
2. Persistencia de datos en tiempo real con Amazon Timestream: Hemos implementado la escritura diferenciada de dos flujos de datos (cotizaciones en crudo y VWAP agregado) en tablas separadas de Timestream, aprovechando las capacidades del servicio para almacenar y consultar series temporales procedentes de dos flujos diferenciados: cotizaciones en crudo y VWAP agregado.
3. Visualización en tiempo real con Grafana: Hemos conectado Grafana con Amazon Timestream mediante el conector nativo y construido un dashboard que permite monitorizar en tiempo real las cotizaciones persistidas y su VWAP.
4. Diseño unificado del consumidor: La decisión de implementar un único script que gestiona ambos topics en lugar de dos scripts independientes simplifica el despliegue, reduce la duplicidad de código y centraliza la lógica de escritura en Timestream, con un mecanismo de captura de excepciones que garantiza la continuidad del consumo ante errores puntuales.
5. Cierre del proyecto TradeData: Este sprint completa el objetivo del proyecto, que ha construido a lo largo de seis sprints una plataforma de análisis de criptomonedas con dos pipelines complementarios: uno batch, que procesa datos históricos de SOLUSD con indicadores técnicos visualizados en QuickSight, y uno streaming, que ingesta datos de mercado en tiempo real, calcula el VWAP y los visualiza en Timestream y Grafana.